
GUIDE 05

PromptFoo for A/B Testing Prompts

Set up, configure, and run systematic prompt experiments with PromptFoo

Agentic AI Builder Expert Bootcamp

Batch 4.0 | April 2026

Table of Contents

1. What is PromptFoo?
2. Installation and Setup
3. Core Concepts
4. Your First A/B Test
5. Writing Test Cases
6. Configuring Providers (Models)
7. Assertion Types and Evaluators
8. LLM-as-Judge with PromptFoo
9. Running and Analyzing Results
10. Advanced: Multi-Model Comparison
11. Advanced: RAG Evaluation
12. CI/CD Integration
13. Best Practices

1. What is PromptFoo?

PromptFoo is an open-source tool for testing and evaluating LLM prompts. It allows you to systematically compare prompt variations across multiple models, test cases, and evaluation criteria -- essentially A/B testing for prompts.

Why PromptFoo?

- **Prompt comparison:** Test multiple prompt versions side-by-side against the same inputs.
- **Model comparison:** Run the same prompt across GPT-4o, Claude, Gemini, etc.
- **Automated evaluation:** Define pass/fail criteria with assertions and LLM judges.
- **Visual dashboard:** Web UI to compare results, spot patterns, and share with team.
- **CI/CD integration:** Run prompt evaluations in your CI pipeline to catch regressions.
- **Cost tracking:** See the cost of each prompt variation across all test cases.

2. Installation and Setup

```
# Install PromptFoo globally
npm install -g promptfoo

# Or use npx (no install needed)
npx promptfoo@latest init

# Verify installation
promptfoo --version
```

Initialize a Project

```
mkdir prompt-experiments && cd prompt-experiments
promptfoo init

# This creates:
# - promptfooconfig.yaml (main config)
# - prompts/              (prompt templates)
```

Set API Keys

```
export OPENAI_API_KEY=sk-xxx
export ANTHROPIC_API_KEY=sk-ant-xxx

# Or create .env file
echo "OPENAI_API_KEY=sk-xxx" > .env
echo "ANTHROPIC_API_KEY=sk-ant-xxx" >> .env
```

3. Core Concepts

Concept	Description	Example
Prompt	The template you're testing	"Summarize this: {{text}}"
Provider	The LLM model to use	openai:gpt-4o, anthropic:claude-sonnet-4-20250514
Test Case	An input + expected output pair	{text: 'long article', expected: 'summary'}
Assertion	A pass/fail check on the output	contains: 'key point', cost < 0.01
Evaluator	Logic that scores the output	LLM-as-judge, regex match, JS function
Variable	Dynamic values inserted into prompts	{{text}}, {{style}}, {{audience}}

4. Your First A/B Test

Let's compare two prompt variations for a summarization task.

Step 1: Create Prompt Files

```
# prompts/v1_basic.txt
Summarize the following text in 2-3 sentences:

{{text}}

# prompts/v2_structured.txt
You are an expert summarizer. Analyze the following text
and provide:
1. A one-sentence summary
2. Key takeaways (2-3 bullet points)
3. Sentiment: positive/negative/neutral

Text: {{text}}
```

Step 2: Configure promptfooconfig.yaml

```
# promptfooconfig.yaml
description: "Summarization A/B Test"

prompts:
  - file://prompts/v1_basic.txt
  - file://prompts/v2_structured.txt

providers:
  - openai:gpt-4o-mini

tests:
  - vars:
      text: "Artificial intelligence has transformed..."
    assert:
      - type: contains
        value: "AI"
```

```

- type: llm-rubric
  value: "Is this a good summary? Score 1-10."

- vars:
  text: "The stock market experienced..."
  assert:
    - type: contains
      value: "market"
    - type: javascript
      value: "output.length < 500"

```

Step 3: Run the Test

```

promptfoo eval

# Open the web dashboard
promptfoo view

```

5. Writing Test Cases

5.1 Inline Test Cases

```

tests:
- vars:
  question: "What is machine learning?"
  assert:
    - type: contains
      value: "data"
    - type: not-contains
      value: "I don't know"
    - type: javascript
      value: "output.length > 50 && output.length < 1000"

- vars:
  question: "Explain quantum computing to a 5 year old"
  assert:
    - type: llm-rubric
      value: "Is this explanation simple enough for a child?"
    - type: not-contains
      value: "quantum entanglement" # Too complex

```

5.2 External Test Cases (CSV)

```

# test_cases.csv
text,expected_sentiment,expected_topic
"Great product, love it!",positive,product
"Terrible service, never again",negative,service
"The weather is mild today",neutral,weather

# In promptfooconfig.yaml
tests: file://test_cases.csv

```

5.3 External Test Cases (JSON)

```
// test_cases.json
[
  {
    "vars": {"text": "Great product!"},
    "assert": [
      {"type": "contains", "value": "positive"},
      {"type": "cost", "threshold": 0.01}
    ]
  }
]
```

6. Configuring Providers (Models)

PromptFoo supports 50+ LLM providers. Here's how to configure the most common ones for A/B testing across models.

```
providers:
  # OpenAI
  - id: openai:gpt-4o
    config:
      temperature: 0
      max_tokens: 1000

  - id: openai:gpt-4o-mini
    config:
      temperature: 0
      max_tokens: 1000

  # Anthropic
  - id: anthropic:messages:claude-sonnet-4-20250514
    config:
      temperature: 0
      max_tokens: 1000

  # Azure OpenAI
  - id: azureopenai:chat:my-gpt4-deployment
    config:
      apiHost: myresource.openai.azure.com

  # Local / Ollama
  - id: ollama:llama3
    config:
      apiBaseUrl: http://localhost:11434
```

7. Assertion Types and Evaluators

Assertion Type	Description	Example
contains	Output contains string	type: contains, value: "AI"
not-contains	Output does not contain	type: not-contains, value: "error"

Assertion Type	Description	Example
equals	Exact match	type: equals, value: "42"
starts-with	Output starts with	type: starts-with, value: "Sure"
regex	Regex pattern match	type: regex, value: "\\d{4}"
javascript	Custom JS function	type: javascript, value: "output.length < 500"
python	Custom Python function	type: python, value: "file://eval.py"
llm-rubric	LLM judges the output	type: llm-rubric, value: "Is this helpful?"
similar	Semantic similarity	type: similar, value: "expected text", threshold: 0.8
cost	Cost below threshold	type: cost, threshold: 0.05
latency	Response time limit	type: latency, threshold: 5000
is-json	Output is valid JSON	type: is-json

8. LLM-as-Judge with PromptFoo

```
# Using llm-rubric (simplest)
tests:
  - vars:
      question: "Explain Docker"
    assert:
      - type: llm-rubric
        value: |
          Rate the explanation on these criteria (1-10):
          1. Accuracy: Are technical details correct?
          2. Clarity: Is it easy to understand?
          3. Completeness: Does it cover key concepts?
          Fail if any score is below 6.

# Using model-graded-closedqa (factual)
  - vars:
      question: "What year was Python created?"
    assert:
      - type: model-graded-closedqa
        value: "Python was created in 1991"

# Custom judge provider
defaultTest:
  options:
    provider:
      id: openai:gpt-4o
      config:
        temperature: 0
```

9. Running and Analyzing Results

Run Commands

```
# Basic run
promptfoo eval

# Run specific config
promptfoo eval -c experiments/summarization.yaml

# Run with specific output format
promptfoo eval -o results.json
promptfoo eval -o results.csv

# Run with table output in terminal
promptfoo eval --table

# Share results
promptfoo share
```

Web Dashboard

```
# Launch interactive dashboard
promptfoo view

# Opens browser at http://localhost:15500
# Dashboard shows:
# - Side-by-side prompt comparison
# - Pass/fail rates per prompt variant
# - Cost breakdown per variant
# - Individual test case results
# - Assertion details and failures
```

Interpreting Results

- **Pass rate:** The percentage of test cases where all assertions passed. Higher is better.
- **Cost:** Total token cost for each prompt variant. Compare cost-effectiveness.
- **Latency:** Average response time. Shorter prompts are usually faster.
- **Failure patterns:** Look at which test cases fail for each variant. This reveals strengths/weaknesses.

10. Advanced: Multi-Model Comparison

```
# Compare same prompt across 4 models
description: "Model Comparison for Agent Router"

prompts:
  - file://prompts/router_prompt.txt

providers:
  - id: openai:gpt-4o
    label: "GPT-4o"
  - id: openai:gpt-4o-mini
```



```

    label: "GPT-4o-mini"
  - id: anthropic:messages:claude-sonnet-4-20250514
    label: "Claude Sonnet"
  - id: anthropic:messages:claude-haiku-4-5-20251001
    label: "Claude Haiku"

tests:
  - vars:
      user_input: "Book a flight to Tokyo"
    assert:
      - type: is-json
      - type: javascript
        value: |
          const parsed = JSON.parse(output);
          parsed.intent === "booking" && parsed.destination === "Tokyo"
      - type: cost
        threshold: 0.02
      - type: latency
        threshold: 3000

```

11. Advanced: RAG Evaluation

```

# Evaluate RAG pipeline with PromptFoo
providers:
  - id: python:rag_provider.py
    label: "RAG Pipeline"

# rag_provider.py
def call_api(prompt, options, context):
    from rag_pipeline import query_rag
    result = query_rag(prompt)
    return {
        "output": result["answer"],
        "metadata": {
            "sources": result["sources"],
            "num_chunks": result["num_chunks"]
        }
    }

tests:
  - vars:
      question: "What is our refund policy?"
    assert:
      - type: contains
        value: "30 days"
      - type: llm-rubric
        value: "Is the answer grounded in the source documents?"
      - type: javascript
        value: |
          const meta = JSON.parse(context.metadata);
          meta.num_chunks >= 1 && meta.num_chunks <= 5

```

12. CI/CD Integration

```
# .github/workflows/prompt-test.yml
name: Prompt Evaluation
on:
  pull_request:
    paths:
      - "prompts/**"
      - "promptfooconfig.yaml"

jobs:
  eval:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: Install PromptFoo
        run: npm install -g promptfoo

      - name: Run Evaluations
        run: promptfoo eval --ci
        env:
          OPENAI_API_KEY: ${ secrets.OPENAI_API_KEY }

      - name: Check Pass Rate
        run: |
          RESULTS=$(promptfoo eval -o json)
          PASS_RATE=$(echo $RESULTS | jq '.stats.passRate')
          if (( $(echo "$PASS_RATE < 0.9" | bc -l) )); then
            echo "Pass rate too low: $PASS_RATE"
            exit 1
          fi
```

13. Best Practices

- **Start small:** Begin with 10-20 test cases. Expand as you learn what matters.
- **Use temperature 0:** Set temperature to 0 for reproducible A/B tests.
- **Version your configs:** Commit promptfooconfig.yaml and prompt files to git.
- **Test one variable at a time:** Change only the prompt OR the model in each experiment.
- **Include edge cases:** Test with empty inputs, very long inputs, adversarial inputs.
- **Track costs:** Use cost assertions to ensure cheaper prompts are not sacrificing quality.
- **Automate in CI:** Block PR merges if prompt evaluations regress below threshold.
- **Use semantic similarity:** Prefer 'similar' assertions over 'equals' for natural language outputs.
- **Document decisions:** When a prompt variant wins, document WHY in a decision log.